

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 10 – SEARCHING



NAMA : Yazid Yaskur Muhammad

NIM : 109082500078

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Searching atau pencarian merupakan proses menemukan suatu data tertentu di dalam kumpulan data. Berbeda dengan pencarian nilai ekstrim yang selalu berhasil menemukan hasil, algoritma searching memiliki dua kemungkinan hasil yaitu data ditemukan atau tidak ditemukan. Salah satu algoritma pencarian yang paling dasar adalah Sequential Search, yaitu algoritma yang memeriksa setiap elemen array satu per satu secara berurutan mulai dari indeks pertama hingga data ditemukan atau seluruh data telah diperiksa. Proses pencarian akan berhenti lebih awal (*early exit*) begitu data yang dicari ditemukan, sehingga tidak perlu memeriksa semua elemen yang tersisa. Algoritma ini dapat diterapkan pada array dalam kondisi apapun, baik terurut maupun tidak terurut, namun memiliki kompleksitas waktu $O(n)$ pada kasus terburuk.

Selain sequential search, terdapat algoritma yang lebih efisien bernama Binary Search, yang bekerja dengan memanfaatkan keterurutan data. Algoritma ini bekerja dengan cara membagi rentang pencarian menjadi dua bagian pada setiap iterasi: diambil elemen tengah dari rentang yang aktif, kemudian dibandingkan dengan nilai yang dicari. Jika nilai tengah terlalu kecil maka rentang pencarian digeser ke kanan, jika terlalu besar digeser ke kiri, dan jika sama maka data ditemukan. Proses ini terus berulang hingga data ditemukan atau rentang pencarian habis ($kr > kn$). Karena setiap iterasi memangkas setengah dari data yang tersisa, binary search memiliki kompleksitas waktu $O(\log n)$ yang jauh lebih efisien dibandingkan sequential search, namun dengan syarat mutlak bahwa data harus sudah dalam keadaan terurut dan algoritma yang digunakan harus sesuai dengan jenis keterurutannya (*ascending* atau *descending*).

B. Guided

1. Guided 1

```
package main

import "fmt"

type arrData [5]string //tipe data alias

func seqSearch(arr arrData, binatangCari string) int {
    var found int = -1
    for i := 0; i < len(arr); i++ {
        if arr[i] == binatangCari {
            found = i
            break
        }
    }
    return found
}

func main() {
    var arrBinatang arrData //array tipe string

    for i := 0; i < len(arrBinatang); i++ {
        fmt.Printf("Masukkan data binatang indeks ke-
%d : ", i)
        fmt.Scan(&arrBinatang[i])
    }
    fmt.Println()

    var binatangCari string
    fmt.Print("Masukkan nama binatang yang mau dicari
: ")
    fmt.Scan(&binatangCari)

    var idxCari int
    idxCari = seqSearch(arrBinatang, binatangCari)

    if idxCari > -1 {
        fmt.Printf("Data %s ditemukan pada indeks ke-
%d!", binatangCari, idxCari)
    }
}
```

```

        } else if idxCari == -1 {
            fmt.Printf("Data %s tidak ditemukan!",
binatangCari)
        }
    }
}

```

Output :

```

PS C:\Users\Asus\Documents\AlproPraktikum\1090825000778_Yazid-Yasku
- Modul 12\Guided\Guided 1\n01.go"
Masukkan data binatang indeks ke-0 : kelinci
Masukkan data binatang indeks ke-1 : kucing
Masukkan data binatang indeks ke-2 : burung
Masukkan data binatang indeks ke-3 : kuda
Masukkan data binatang indeks ke-4 : hamster

Masukkan nama binatang yang mau dicari : kucing
Data kucing ditemukan pada indeks ke-1!

```

2. Guided 2

```

package main

import "fmt"

type arrData [5]string //tipe data alias

func seqSearch(arr arrData, binatangCari string) int {
    var found int = -1
    for i := 0; i < len(arr); i++ {
        if arr[i] == binatangCari {
            found = i
            break
        }
    }
    return found
}

func main() {
    var arrBinatang arrData //array tipe string

```

```

        for i := 0; i < len(arrBinatang); i++ {
            fmt.Printf("Masukkan data binatang indeks ke-
%d : ", i)
            fmt.Scan(&arrBinatang[i])
        }
        fmt.Println()

        var binatangCari string
        fmt.Print("Masukkan nama binatang yang mau dicari
: ")
        fmt.Scan(&binatangCari)

        var idxCari int
        idxCari = seqSearch(arrBinatang, binatangCari)

        if idxCari > -1 {
            fmt.Printf("Data %s ditemukan pada indeks ke-
%d!", binatangCari, idxCari)
        } else if idxCari == -1 {
            fmt.Printf("Data %s tidak ditemukan!",
binatangCari)
        }
    }
}

```

Output :

```

PS C:\Users\Asus\Documents\AlproPraktikum\1090825000778_Yazid-Yaskur-Muhammad
- Modul 12\Guided\Guided 1\n01.go"
Masukkan data binatang indeks ke-0 : kuda
Masukkan data binatang indeks ke-1 : hamster
Masukkan data binatang indeks ke-2 : kucing
Masukkan data binatang indeks ke-3 : kelinci
Masukkan data binatang indeks ke-4 : burung

Masukkan nama binatang yang mau dicari : burung
Data burung ditemukan pada indeks ke-4!
PS C:\Users\Asus\Documents\AlproPraktikum\1090825000778_Yazid-Yaskur-Muhammad
- Modul 12\Guided\Guided 1\n01.go"
Masukkan data binatang indeks ke-0 : kuda
Masukkan data binatang indeks ke-1 : hamster
Masukkan data binatang indeks ke-2 : kucing
Masukkan data binatang indeks ke-3 : kelinci
Masukkan data binatang indeks ke-4 : burung

Masukkan nama binatang yang mau dicari : ikan
Data ikan tidak ditemukan!
PS C:\Users\Asus\Documents\AlproPraktikum\1090825000778_Yazid-Yaskur-Muhammad

```

3. Guided 3

```
package main

import "fmt"

type mahasiswa struct {
    nama string
    nim   int
    IPK   float64
}

type arrMahasiswa [5]mahasiswa

func binSearchStruct(arrData arrMahasiswa, nimDicari
int) int {
    var found int = -1
    var med int
    var kr int = 0
    var kn int = len(arrData) - 1
    for kr <= kn && found == -1 {
        med = (kr + kn) / 2
        if nimDicari < arrData[med].nim {
            kn = med - 1
        } else if nimDicari > arrData[med].nim {
            kr = med + 1
        } else {
            found = med
        }
    }
    return found
}

func seqSearchStruct(arrData arrMahasiswa, nimDicari
int) int {
    var found int = -1
    for i := 0; i < len(arrData); i++ {
        if arrData[i].nim == nimDicari {
            found = i
            break
        }
    }
    return found
}
```

```

func main() {
    var mahasiswa06 arrMahasiswa
    var nimDicari int

    fmt.Println("--- MASUKKAN DATA NIM MAHASISWA
SECARA TERURUT ---")
    for i := 0; i < len(mahasiswa06); i++ {
        fmt.Printf("=== Masukkan data mahasiswa 06
pada indeks ke-%d ===\n", i)
        fmt.Print("Masukkan nama : ")
        fmt.Scan(&mahasiswa06[i].nama)
        fmt.Print("Masukkan NIM : ")
        fmt.Scan(&mahasiswa06[i].nim)
        fmt.Print("Masukkan IPK : ")
        fmt.Scan(&mahasiswa06[i].IPK)
        fmt.Println("-----")
    }
    fmt.Println()

    fmt.Print("Masukkan NIM mahasiswa yang ingin
dicari : ")
    fmt.Scan(&nimDicari)
    fmt.Println()

    var idxHasilCariSeqSearch int
    idxHasilCariSeqSearch =
seqSearchStruct(mahasiswa06, nimDicari)

    fmt.Println("== HASIL SEQUENTIAL SEARCH ==")
    if idxHasilCariSeqSearch > -1 {
        fmt.Printf("Data NIM mahasiswa %d ditemukan
pada array di indeks ke-%d!\n", nimDicari,
idxHasilCariSeqSearch)
        fmt.Println("Berikut Detail Datanya : ")
        fmt.Printf("Nama : %s\n",
mahasiswa06[idxHasilCariSeqSearch].nama)
        fmt.Printf("NIM : %d\n",
mahasiswa06[idxHasilCariSeqSearch].nim)
        fmt.Printf("IPK : %.2f\n",
mahasiswa06[idxHasilCariSeqSearch].IPK)
    } else if idxHasilCariSeqSearch == -1 {

```

```

        fmt.Printf("Data NIM mahasiswa %d tidak
ditemukan pada array!", nimDicari)
    }
    fmt.Println()

    var idxHasilCariBinSearch int
    idxHasilCariBinSearch =
binSearchStruct(mahasiswa06, nimDicari)

    fmt.Println("== HASIL BINARY SEARCH ==")
    if idxHasilCariBinSearch > -1 {
        fmt.Printf("Data NIM mahasiswa %d ditemukan
pada array di indeks ke-%d!\n", nimDicari,
idxHasilCariBinSearch)
        fmt.Println("Berikut Detail Datanya : ")
        fmt.Printf("Nama : %s\n",
mahasiswa06[idxHasilCariBinSearch].nama)
        fmt.Printf("NIM : %d\n",
mahasiswa06[idxHasilCariBinSearch].nim)
        fmt.Printf("IPK : %.2f\n",
mahasiswa06[idxHasilCariBinSearch].IPK)
    } else if idxHasilCariBinSearch == -1 {
        fmt.Printf("Data NIM mahasiswa %d tidak
ditemukan pada array!", nimDicari)
    }
}

```

Output:

```

=== Masukkan data mahasiswa 06 pada indeks ke-0 ===
Masukkan nama : Yaskur
Masukkan NIM : 109082500078
Masukkan IPK : 3.5
-----
=== Masukkan data mahasiswa 06 pada indeks ke-1 ===
Masukkan nama : Erpan
Masukkan NIM : 109082500079
Masukkan IPK : 3.4
-----
=== Masukkan data mahasiswa 06 pada indeks ke-2 ===
Masukkan nama : Faig
Masukkan NIM : 109082500077
Masukkan IPK : 3.3
-----
=== Masukkan data mahasiswa 06 pada indeks ke-3 ===
Masukkan nama : Arsyia
Masukkan NIM : 109082500076
Masukkan IPK : 3.2
-----
=== Masukkan data mahasiswa 06 pada indeks ke-4 ===
Masukkan nama : Hanif
Masukkan NIM : 109082500075
Masukkan IPK : 3.1
-----

Masukkan NIM mahasiswa yang ingin dicari : 109082500078

== HASIL SEQUENTIAL SEARCH ==
Data NIM mahasiswa 109082500078 ditemukan pada array di indeks ke-0!
Berikut Detail Datanya :
Nama : Yaskur
NIM : 109082500078
IPK : 3.50

```

C. Unguided

1. Soal Unguided 1

Pada pemilihan ketua RT yang baru saja berlangsung, terdapat 20 calon ketua yang bertanding memperebutkan suara warga. Perhitungan suara dapat segera dilakukan karena warga cukup mengisi formulir dengan nomor dari calon ketua RT yang dipilihnya. Seperti biasa, selalu ada pengisian yang tidak tepat atau dengan nomor pilihan di luar yang tersedia, sehingga data juga harus divalidasi. Tugas Anda untuk membuat program mencari siapa yang memenangkan pemilihan ketua RT.

```
package main

import "fmt"

func main() {
    var suara int
    var totalMasuk int = 0
    var totalSah int = 0
    var hitungSuara [21]int

    for {
        fmt.Scan(&suara)
        if suara == 0 {
            break
        }
        totalMasuk++
        if suara >= 1 && suara <= 20 {
            totalSah++
            hitungSuara[suara]++
        }
    }

    fmt.Printf("Suara masuk: %d\n", totalMasuk)
    fmt.Printf("Suara sah: %d\n", totalSah)

    for i := 1; i <= 20; i++ {
        if hitungSuara[i] > 0 {
            fmt.Printf("%d: %d\n", i, hitungSuara[i])
        }
    }
}
```

```
}
```

Output :

```
PS C:\Users\Asus\Documents\AlproPraktikum\1090825000778_Y  
nerFile.go"  
7 19 3 2 78 3 1 -3 18 19 0  
Suara masuk: 10  
Suara sah: 8  
1: 1  
2: 1  
3: 2  
7: 1  
18: 1  
19: 2
```

Penjelasan :

Program ini bertujuan membaca, memvalidasi, dan menghitung suara pemilihan ketua RT menggunakan array `hitungSuara[21]int` sebagai media penyimpanan perolehan suara, di mana indeks array langsung merepresentasikan nomor calon (1–20) sehingga `hitungSuara[3]` berarti jumlah suara untuk calon nomor 3. Input dibaca secara terus-menerus menggunakan loop tanpa kondisi awal dan baru berhenti ketika menemukan angka 0 sebagai penanda akhir data, setiap angka yang terbaca (kecuali 0) langsung diakumulasi ke `totalMasuk`, kemudian divalidasi apakah nilainya berada di rentang 1–20 inklusif — jika valid maka `totalSah` bertambah dan `hitungSuara[suara]` di-increment, jika tidak valid (seperti 78 atau -3) maka diabaikan. Setelah semua input selesai dibaca, program mencetak `totalMasuk` dan `totalSah`, lalu melakukan loop dari indeks 1 hingga 20 untuk menampilkan perolehan suara hanya untuk calon yang mendapatkan suara (> 0), sehingga urutan tampilan otomatis dari nomor calon terkecil ke terbesar.

2. Soal Unguided 2

Berdasarkan program sebelumnya, buat program `pilkart` yang mencari siapa pemenang pemilihan ketua RT. Sekaligus juga ditentukan bahwa wakil ketua RT adalah calon yang mendapatkan suara terbanyak kedua. Jika beberapa calon mendapatkan suara terbanyak yang sama, ketua terpilih adalah dengan nomor peserta yang paling kecil dan wakilnya dengan nomor peserta terkecil berikutnya

Jawaban :

```
package main  
  
import "fmt"
```

```

func cariKetuaWakil(hitungSuara [21]int) (int, int) {
    // Cari suara terbanyak pertama dan kedua
    // Jika sama, pilih nomor peserta terkecil
    var ketua int = -1
    var wakil int = -1

    for i := 1; i <= 20; i++ {
        if hitungSuara[i] > 0 {
            if ketua == -1 {
                ketua = i
            } else if hitungSuara[i] >
hitungSuara[ketua] {
                wakil = ketua
                ketua = i
            } else if wakil == -1 {
                wakil = i
            } else if hitungSuara[i] >
hitungSuara[wakil] {
                wakil = i
            }
        }
    }
    return ketua, wakil
}

func main() {
    var suara int
    var totalMasuk int = 0
    var totalSah int = 0
    var hitungSuara [21]int

    for {
        fmt.Scan(&suara)
        if suara == 0 {
            break
        }
        totalMasuk++
        if suara >= 1 && suara <= 20 {
            totalSah++
            hitungSuara[suara]++
        }
    }
}

```

```

    }

    fmt.Printf("Suara masuk: %d\n", totalMasuk)
    fmt.Printf("Suara sah: %d\n", totalSah)

    ketua, wakil := cariKetuaWakil(hitungSuara)
    fmt.Printf("Ketua RT: %d\n", ketua)
    fmt.Printf("Wakil ketua: %d\n", wakil)
}

```

Output :

```

PS C:\Users\Asus\Documents\AlproPraktikum\1090825000778_Yazid-Yaskur-
nerFile.go"
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 10
Suara sah: 8
Ketua RT: 3
Wakil ketua: 19

```

Penjelasan :

Program ini merupakan pengembangan dari soal sebelumnya yang ditambahkan fungsi cariKetuaWakil untuk menentukan ketua dan wakil ketua RT berdasarkan perolehan suara terbanyak. Setelah proses pembacaan dan validasi input yang sama seperti sebelumnya, fungsi cariKetuaWakil melakukan iterasi array hitungSuara dari indeks 1 hingga 20 untuk membandingkan perolehan suara tiap calon — variabel ketua menyimpan indeks calon dengan suara terbanyak dan wakil menyimpan indeks calon dengan suara terbanyak kedua, di mana jika calon baru memiliki suara lebih banyak dari ketua maka ketua lama digeser menjadi wakil dan ketua diperbarui, sedangkan jika suara calon baru lebih banyak dari wakil maka hanya wakil yang diperbarui. Aturan tie-breaking (suara sama, pilih nomor terkecil) terpenuhi secara otomatis karena iterasi dilakukan dari nomor 1 ke 20, sehingga calon dengan nomor lebih kecil akan lebih dulu tersimpan di ketua atau wakil dan tidak akan tergantikan kecuali ada yang benar-benar lebih banyak suaranya.

3. Soal Unguided 3

Diberikan n data integer positif dalam keadaan terurut membesar dan sebuah integer lain k, apakah bilangan k tersebut ada dalam daftar bilangan yang diberikan? Jika ya, berikan indeksnya, jika tidak sebutkan "TIDAK ADA".

Jawaban :

```
package main

import "fmt"

const NMAX = 1000000

var data [NMAX]int

func main() {
    var n, k int
    fmt.Scan(&n, &k)
    isiArray(n)
    hasil := posisi(n, k)
    if hasil == -1 {
        fmt.Println("TIDAK ADA")
    } else {
        fmt.Println(hasil)
    }
}

func isiArray(n int) {
    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }
}

func posisi(n, k int) int {
    var found int = -1
    var kr int = 0
    var kn int = n - 1
    var med int

    for kr <= kn && found == -1 {
        med = (kr + kn) / 2
        if k < data[med] {
            kn = med - 1
        } else if k > data[med] {
            kr = med + 1
        } else {
            found = med
        }
    }
}
```

```

    }
}
return found
}

```

Output :

```

PS C:\Users\Asus\Documents\AlproPraktikum\1090825000778_Yazid-Yaskur-Muhammad>
3\no3.go"
12 534
1 3 8 16 32 123 323 323 534 543 823 999
8
PS C:\Users\Asus\Documents\AlproPraktikum\1090825000778_Yazid-Yaskur-Muhammad>
3\no3.go"
12 535
1 3 8 16 32 123 323 323 534 543 823 999
TIDAK ADA

```

Penjelasan :

Program ini bertujuan mencari posisi bilangan k dalam array berisi n data integer positif yang sudah terurut membesar menggunakan algoritma **binary search**. Program dibagi menjadi tiga subprogram sesuai kerangka yang diminta: main bertugas membaca n dan k dari baris pertama lalu mencetak hasil akhir, prosedur isiArray(n) mengisi array global data dengan membaca n bilangan dari input, dan fungsi posisi(n, k) mengimplementasikan binary search dengan dua pointer kr (kiri) dan kn (kanan) yang mengapit rentang pencarian — setiap iterasi diambil elemen tengah med, jika k lebih kecil dari data[med] maka kn digeser ke med-1, jika lebih besar maka kr digeser ke med+1, dan jika sama maka indeks med langsung disimpan sebagai found dan loop berhenti. Proses pencarian berakhir ketika data ditemukan atau rentang habis (kr > kn), dan fungsi mengembalikan indeks posisi data (berbasis 0) jika ditemukan atau -1 jika tidak ditemukan, yang kemudian di main diterjemahkan menjadi output angka indeks atau teks "TIDAK ADA".

D. Kesimpulan

Modul 12 membahas algoritma pencarian (*searching*) yang digunakan untuk menemukan data spesifik dalam suatu kumpulan data, di mana hasil pencarian memiliki dua kemungkinan yaitu ditemukan atau tidak ditemukan. Terdapat dua algoritma utama yang dipelajari, yaitu Sequential Search yang bekerja dengan memeriksa setiap elemen satu per satu dari awal hingga data ditemukan atau seluruh elemen habis diperiksa — algoritma ini sederhana dan dapat diterapkan pada data dalam kondisi apapun (terurut maupun tidak) namun kurang efisien untuk data berukuran besar karena

kompleksitasnya $O(n)$, serta Binary Search yang jauh lebih efisien dengan kompleksitas $O(\log n)$ karena memanfaatkan keterurutan data untuk memangkas setengah rentang pencarian di setiap iterasinya, namun dengan syarat mutlak data harus sudah terurut dan jenis keterurutannya (*ascending* atau *descending*) harus sesuai dengan algoritma yang digunakan. Kedua algoritma tersebut juga dapat diterapkan pada array bertipe data struct dengan menambahkan field kategori sebagai kunci pencarian, di mana binary search hanya bisa digunakan apabila array terurut berdasarkan field yang sama dengan kunci pencariannya, sedangkan sequential search lebih fleksibel karena bisa mencari berdasarkan field apapun tanpa bergantung pada keterurutan data.